

# Labyrinth Breach – Viva Cheatsheet

AI641 – AI for Robotics | MS AI, LUMS | May 2025

## Project At a Glance

**What:** 3 Sentinels (pursuers) vs 2 Runners (evaders) in a dynamic maze with shifting walls and exit zones.

**Framework:** Unity ML-Agents 4.0.0, Unity 6000.0.40f1

**Algorithm:** PPO with parameter sharing (all 3 Sentinels share one brain, both Runners share another)

**Formulation:** Partially Observable Stochastic Game (POSG)

$\langle N, \mathcal{S}, \{A_i\}, \{O_i\}, T, \{R_i\}, \gamma \rangle$ ,  $N=5$ ,  $\gamma=0.99$

**Win conditions:**

- Sentinels win: capture both Runners before timeout
- Runners win: reach exit OR survive 120s (5000 steps)

## Critical Numbers

| Item                  | Value              |
|-----------------------|--------------------|
| Agents total          | 5 (3+2)            |
| Episode timeout       | 120s / 5000 steps  |
| Sentinel obs size     | 117 floats         |
| Runner obs size       | 129 floats         |
| Action space (both)   | 2 continuous       |
| Sentinel rays         | 14 (84 floats)     |
| Runner rays           | 16 (96 floats)     |
| Floats per ray        | 6                  |
| Memory floats         | 6                  |
| Max training steps    | 500,000            |
| Learning rate         | $3 \times 10^{-4}$ |
| Batch / Buffer        | 1024 / 10240       |
| Hidden units / layers | 256 / 2            |
| Discount $\gamma$     | 0.99               |
| GAE $\lambda$         | 0.95               |
| Clip $\epsilon$       | 0.2                |
| Entropy $\beta$       | $5 \times 10^{-3}$ |
| Time horizon          | 64                 |
| Epochs per update     | 3                  |
| Curriculum stages     | 4                  |
| Min total episodes    | 9,600              |
| Eval seeds            | 42, 101, 202       |
| Seen win rate         | 62.0%              |
| Unseen win rate       | 60.3%              |
| 1st capture (seen)    | 17.2s              |
| 1st capture (unseen)  | 9.7s               |

## Observation Breakdown

| Component                         | Sent.      | Run.       |
|-----------------------------------|------------|------------|
| Self state (pos,vel,head,alive)   | 10         | 10         |
| Env context (time,exit,wall)      | 7          | 7          |
| Rays ( $N \times 6$ )             | 84         | 96         |
| Memory (last-known pos)           | 6          | 6          |
| Opponent summary ( $2 \times 5$ ) | 10         | 10         |
| <b>Total</b>                      | <b>117</b> | <b>129</b> |

**Each ray** = 6 floats: normalized distance + one-hot for wall, exit, teammate, opponent, other.

**Memory** = stores last-known XYZ of target + time since last seen. Activates when LOS breaks.

**Opponent summary** = 2 nearest opponents: distance, relative XZ, LOS flag, pressure.

**Why more Runner rays?** Runners must detect threats from all directions. Sentinels focus forward on pursuit targets.

## Reward Design

### Sentinel Rewards

|                       |         |
|-----------------------|---------|
| Team capture (win)    | +1.0    |
| Timeout/escape (loss) | -1.0    |
| Individual capture    | +0.1    |
| Chase progress/step   | +0.003  |
| Chase regression      | -0.0025 |
| Wall loop penalty     | -0.002  |
| Exploration bonus     | +0.0008 |
| Orbit stall penalty   | -0.0025 |

### Runner Rewards

|                      |         |
|----------------------|---------|
| Team exit (win)      | +1.0    |
| Both captured (loss) | -1.0    |
| Tagged individually  | -0.2    |
| Survival/step        | +0.0005 |
| Evade progress       | +0.0025 |
| Threat approach      | -0.0035 |
| Wall loop penalty    | -0.0015 |
| Exploration bonus    | +0.0012 |
| Orbit stall penalty  | -0.003  |

### Tactical Layer (Optional)

Detected by TrapEventDetector:

- **Pincer:** 2 Sentinels on opposite sides of Runner

- **Corridor control:** blocking both corridor ends
- **Exit denial:** guarding exit while Runner nearby
- **Dead-end forcing:** pushing Runner into dead end
- **Cluster penalty:** discourages Sentinel bunching

## Curriculum (4 Stages)

| Stage               | Ep.  | Walls | WR  |
|---------------------|------|-------|-----|
| Static, fixed spawn | 1200 | —     | .62 |
| Static, rand spawn  | 2200 | —     | .55 |
| Dynamic, low freq   | 3200 | 20s   | .50 |
| Dynamic, high freq  | 3000 | 8s    | .45 |

Agents advance only after hitting min episodes AND win-rate threshold. Total: 9,600+ episodes minimum.

## PPO – Know This Cold

**What:** Policy gradient method by OpenAI. Updates policy in small, controlled steps.

**Objective:**  $L^{CLIP} = \mathbb{E}[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1\pm\epsilon)\hat{A}_t)]$

where  $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$

**Key ideas:**

- Clips ratio to  $[1-\epsilon, 1+\epsilon]$  so updates are never too large
- Uses GAE ( $\lambda$ ) for advantage estimation: balances bias vs variance
- Entropy bonus ( $\beta$ ) encourages exploration early, decays over time
- Linear LR schedule: starts at  $3 \times 10^{-4}$ , decreases to 0

**Why PPO here?**

- Stable with parameter sharing (3 agents, 1 brain)
- Native support in Unity ML-Agents
- Yu et al. (MAPPO) showed PPO is surprisingly strong for MARL when tuned carefully

## MA-POCA vs PPO

**MA-POCA** = Multi-Agent POsthumous Credit Assignment (Unity).

| PPO (ours)                                     | MA-POCA                                    |
|------------------------------------------------|--------------------------------------------|
| Single-agent algo applied to MARL              | Built for multi-agent                      |
| Shared reward to all                           | Central critic evaluates each agent        |
| If agent “dies” mid-episode, no further credit | “Posthumous”: dead agents still get credit |
| Simple, stable                                 | More complex but better credit assignment  |

**If asked “why not MA-POCA?”:** “We use PPO as a stable baseline. MA-POCA is configured in our codebase for future comparison to study whether centralized credit assignment produces measurably better coordination.”

## Key Concepts for Viva

### POSG (Partially Observable Stochastic Game)

Extension of MDP to multiple agents with partial observations. Each agent sees  $O_i$ , not full state  $S$ . Non-stationary because other agents change the environment.

### CTDE (Centralized Training, Decentralized Execution)

During training: a central process sees everything and computes gradients. During inference: each agent runs its own policy from local observations only. Used by MADDPG, MA-POCA.

### Parameter Sharing

All agents of same role share one neural network. Sentinel A, B, C use same weights. Advantage: what one learns, all know. Disadvantage: cannot specialize.

### Reward Shaping

Adding intermediate rewards to guide learning when terminal reward is sparse. Risk: agents may exploit shaping rewards instead of pursuing actual objective.

### Generalization / Transfer

Testing on unseen maze layouts. If win rate holds, the policy learned general strategies, not memorized routes.

### Credit Assignment Problem

With team reward, who gets credit? The Sentinel that blocked the corridor or the one that tagged the Runner? QMIX, COMA, MA-POCA all address this.

### Curriculum Learning

Start with easy tasks, gradually increase difficulty. Helps discover behaviors that are hard to learn from scratch in hard

environments.

## Key Papers (What to Say)

### MADDPG (Lowe 2017)

Centralized critic + decentralized actors for mixed cooperative-competitive. “We use the same CTDE principle but with PPO instead of DDPG.”

### MAPPO (Yu 2022)

PPO is surprisingly effective for cooperative MARL. “This paper validates our choice of PPO as a strong baseline.”

### QMIX (Rashid 2018)

Value factorization for cooperative teams. “We chose policy gradient (PPO) over value decomposition because our action space is continuous.”

### COMA (Foerster 2018)

Counterfactual baseline for credit assignment. “We address credit assignment through reward shaping rather than counterfactual baselines.”

### ViPER (Wang 2025)

Visibility-based pursuit-evasion with RL. “Our work adds team coordination (3v2) and dynamic topology on top of visibility constraints.”

### MatrixWorld (Cheng 2023)

Grid-based PE benchmark. “We operate in continuous 3D space with asymmetric teams and exit objectives.”

## Likely Viva Questions

### Q: What is the state space size?

A: 117 floats (Sentinel), 129 floats (Runner). Difference is ray count: 14 vs 16 rays, 6 floats each.

Show: Scripts/Sensors/ObservationAssembler.cs lines 23–26

### Q: Why PPO and not DQN?

A: DQN is for discrete actions. Our agents output 2 continuous floats (moveX, moveZ). PPO handles continuous action spaces natively.

Show: configs/trainer\_configs/ppo\_openarena\_3v2.yaml

### Q: How do you handle partial observability?

A: Raycasts (not global state), last-known-position memory for targets out of LOS, opponent summary for 2 nearest enemies.

Show: Sensors/RaySensorBuilder.cs, Sensors/LastKnownPositionMemory.cs

### Q: What proves the agents coordinate?

A: Outcome-level evidence: stable win rates on unseen layouts. Qualitative: observed pincer/corridor blocking in play-back. We acknowledge this is not conclusive proof of inten-

tional coordination.

### Q: Why do unseen mazes have faster captures?

A: Likely because unseen layouts have more constrained corridors, limiting Runner escape options. Policies transfer but are sensitive to topology.

### Q: What is the discount factor and why 0.99?

A:  $\gamma = 0.99$  means the agent values future rewards highly. Important because capture may happen many steps after a good positioning move.

### Q: What would you do differently?

A: Run MA-POCA comparison for credit assignment. Add ablation studies (memory on/off, shaping on/off). Compute coordination metrics from step-level logs.

### Q: How do you prevent reward exploitation?

A: Reward audit logs track every reward event per agent per step. Orbit stall and wall loop penalties discourage degenerate behavior.

Show: Rewards/RewardEngine.cs, Output: results/\*/logs/reward\_audit.csv

### Q: What happens when a Runner is captured?

A: Runner is deactivated. Sentinels must continue pursuing remaining Runner. The reward system gives individual capture bonus but no terminal reward until all are captured.

Show: Environment/PursuitEvasionEnvController.cs capture logic

### Q: What is the role of entropy ( $\beta$ )?

A: Entropy bonus encourages the policy to stay stochastic early in training, promoting exploration. Too much entropy = random behavior. Too little = premature convergence.

### Q: Training vs inference – what changes?

A: Observations are identical. During training, Python API collects data and updates weights with exploration. During inference, Unity runs the ONNX model locally with deterministic actions. No Python needed.

Show: unity/Assets/Models/\*.onnx, Prefabs/Sentinel.prefab BehaviorType

### Q: What is GAE ( $\lambda$ )?

A: Generalized Advantage Estimation. Balances bias (low  $\lambda$ , like TD) and variance (high  $\lambda$ , like Monte Carlo). We use  $\lambda = 0.95$ , favoring lower variance.

### Q: Why 4 curriculum stages?

A: Static fixed spawns teach basic pursuit. Random spawns prevent memorization. Low-freq walls introduce adaptation. High-freq walls force real-time replanning. Each stage builds on the last.

Show: configs/curriculum\_configs/curriculum\_3v2\_full\_v1.yaml

## — HIGH PRIORITY (asked in other group’s viva) —

### Q: Why did you evaluate using win rates and cap-

ture times instead of quantifying emergent behavior directly?

A: “We deliberately chose outcome-level metrics (win rates, capture timing, escape fractions) because they are objective and reproducible across seeds. We do observe qualitative coordination in playback, such as pincer formations and corridor blocking. However, our logging system (replay\_events.csv and reward\_audit.csv) already records per-step positions and tactical events. From these logs we could compute direct coordination metrics like pincer frequency, corridor-block rate, inter-Sentinel angular spread, and coverage overlap. We list this as future work in the paper because we wanted to report only metrics we can fully defend.”

Q: What specific metrics would you use to quantify emergent coordination?

A: Four concrete metrics from our logs:

- 1. **Pincer rate:** fraction of captures where 2+ Sentinels were on opposite sides of the Runner (angle > 120°)
- 2. **Coverage spread:** average pairwise distance between Sentinels, measuring whether they fan out vs cluster
- 3. **Corridor block rate:** fraction of time a Sentinel occupies both ends of a corridor segment
- 4. **Exit denial rate:** fraction of time at least one Sentinel is within threat radius of an active exit while a Runner is nearby

All of these can be computed post-hoc from agent\_step\_log.csv positions. Our TrapEventDetector already detects these events during runtime.

File Index – Open Quickly During Viva

All paths relative to project root.

C# Scripts (unity/Assets/Scripts/)

| Concept                           |                            | File                                                                  |
|-----------------------------------|----------------------------|-----------------------------------------------------------------------|
| Episode logic, capture, terminals | Base agent (movement, obs) | Environment/<br>PursuitEvasionEnvController.cs<br>Agents/BaseAgent.cs |
| Sentinel agent                    | Runner agent               | Agents/SentinelAgent.cs<br>Agents/RunnerAgent.cs                      |
| Obs vector assembly               |                            | Sensors/ObservationAssembler.cs                                       |
| 360 raycasts (14/16 rays)         |                            | Sensors/RaySensorBuilder.cs                                           |
| LOS memory                        |                            | Sensors/LastKnownPositionMemory.cs                                    |
| Opponent summary                  |                            | Sensors/ObservationAssembler.cs<br>L139                               |
| Visibility check                  |                            | Sensors/VisibilityTracker.cs                                          |
| Reward engine (all rewards)       |                            | Rewards/RewardEngine.cs                                               |
| Reward config (YAML parse)        |                            | Rewards/RewardConfig.cs                                               |
| Sentinel reward policy            |                            | Rewards/SentinelRewardPolicy.cs                                       |
| Runner reward policy              |                            | Rewards/RunnerRewardPolicy.cs                                         |
| Reward event types                |                            | Rewards/RewardEvent.cs                                                |
| Trap/pincer detection             |                            | Rewards/TrapEventDetector.cs                                          |
| CSV logging (4 files)             |                            | Logging/StepLogger.cs                                                 |

Configs (configs/)

| Concept            |                    | File                                                                                          |
|--------------------|--------------------|-----------------------------------------------------------------------------------------------|
| PPO params         | Reward weights     | trainer_configs/<br>ppo_openarena_3v2.yaml<br>reward_configs/<br>reward_shared_basic_v1.yaml  |
| 4-stage curriculum | Unseen eval config | curriculum_configs/<br>curriculum_3v2_full_v1.yaml<br>env_configs/<br>env_unseen_eval_v1.yaml |

Unity Assets (unity/Assets/)

| Concept           |       | File                                  |
|-------------------|-------|---------------------------------------|
| Sentinel model    | ONNX  | Models/Sentinel_seed101_stage4.onnx   |
| Runner model      | ONNX  | Models/Runner_seed101_stage4.onnx     |
| Sentinel prefab   |       | Prefabs/Sentinel.prefab               |
| Runner prefab     |       | Prefabs/Runner.prefab                 |
| Open scene        | arena | Scenes/01_Baseline_OpenArena_3v2      |
| Static scene      | maze  | Scenes/02_StaticMaze_3v2              |
| Dynamic scene     | maze  | Scenes/03_DynamicMaze_3v2             |
| Unseen scene      | eval  | Scenes/04_Eval_UnseenMaze_3v2         |
| Sentinel material |       | Materials/Sentinel_Red.mat<br>(blue!) |
| Runner material   |       | Materials/Runner_Blue.mat<br>(pink!)  |

Scripts and Outputs

| Concept               |      | File                                 |
|-----------------------|------|--------------------------------------|
| Eval/inference script |      | scripts/evaluate_policy.py           |
| Setup validation      |      | scripts/setup_validation.py          |
| Episode outcomes      | out- | results/*/logs/episode_log.csv       |
| Per-step positions    |      | results/*/logs/agent_step_log.csv    |
| Reward audit trail    |      | results/*/logs/reward_audit.csv      |
| Replay events         |      | results/*/logs/replay_events.csv     |
| Coordination metrics  |      | results/*/kpis/coordination_metrics. |

Data Flow (memorize this)

YAML config → RewardConfig struct → Policy class → RewardEngine → per-agent AddReward() → StepLogger → CSV files

Material name trap: Sentinel\_Red.mat is blue. Runner\_Blue.mat is pink. Legacy names, do not rename (GUID references).

Results Summary

| Metric             | Seen  | Unseen |
|--------------------|-------|--------|
| Sentinel win rate  | 0.620 | 0.603  |
| Runner win rate    | 0.380 | 0.397  |
| 1st capture (s)    | 17.15 | 9.68   |
| 2nd capture (s)    | 37.84 | 24.55  |
| Runner escape frac | 0.260 | 0.268  |
| Timeout survival   | 0.120 | 0.129  |

**Key takeaway:** Win rates are stable (1.7pt drop). Capture timing shifts significantly (−7.5s first, −13.3s second). Policies transfer but are topology-sensitive.

**Safe claim:** “The learned policies transfer across maze layouts while remaining sensitive to topology.”

**Unsafe claim (avoid):** “The agents have learned spatial reasoning” or “The agents intentionally coordinate.”